

Experiment 'p_13_9_recursive_sum_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_9_recursive_sum_array

Natural language statement: null

Method signature: p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs[1..])`

Functional Code Given

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)`

The method should respect the following contract:

`ensures sum == sum_array(inputs[..])`

The contract uses the following dafny code:

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSOOZt1MpGn3fZ8ftlv4ibjtYdmK

Model: gpt-5.5-2026-04-23

Created at: 1780679584

Finish reason: stop

Usage: 436 tokens in, and 3871 tokens out

```
{
  sum := 0;
  var i := inputs.Length;
  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant sum == sum_array(inputs[i..])
    decreases i
  {
    var oldI := i;
```

```

    i := i - 1;
    assert oldI == i + 1;
    assert sum == sum_array(inputs[oldI..]);
    assert sum == sum_array(inputs[i + 1..]);
    assert inputs[i..] == [inputs[i]] + inputs[i + 1..];
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];
    assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);
    sum := inputs[i] + sum;
    assert sum == sum_array(inputs[i..]);
  }
  assert i == 0;
  assert inputs[i..] == inputs[..];
}

```

Final Program

```

function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)
  ensures sum == sum_array(inputs[..])
{
  sum := 0;
  var i := inputs.Length;
  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant sum == sum_array(inputs[i..])
    decreases i
  {
    var oldI := i;
    i := i - 1;
    assert oldI == i + 1;
    assert sum == sum_array(inputs[oldI..]);
    assert sum == sum_array(inputs[i + 1..]);
    assert inputs[i..] == [inputs[i]] + inputs[i + 1..];
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];
    assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);
    sum := inputs[i] + sum;
    assert sum == sum_array(inputs[i..]);
  }
  assert i == 0;
  assert inputs[i..] == inputs[..];
}

```

Total Token Usage

Input tokens: 436

Output tokens: 3871

Reasoning tokens: 3584

Sum of 'total tokens': 4307

Experiment Timings

Overall Experiment started at 1780679583763, ended at 1780679714155, lasting 130392ms (130.39 seconds)

Iteration #1 started at 1780679583764, ended at 1780679714155, lasting 130391ms (130.39 seconds)